Міністерство освіти і науки України
Національний технічний університет України
Київський політехнічний інститут імені Ігоря Сікорського

ЛАБОРАТОРНА РОБОТА №3

з дисципліни «Архітектура комп’ютерів 2»

на тему «Розвертання програмного забезпечення»

ВИКОНАЛА:
студентка групи ІО-44
Копилець Дар’я
ПЕРЕВІРИВ:
Каплунов А.В.

Київ 2026

Мета: Робота з інструкцією "BeagleBone Black: Platform Bring Up with Upstream Components", опрацьовування частини завдання "Збирання виконуваних файлів для ОС Linux для процесорів архітектури ARM."

Завдання: Розвернути програмне середовище, підготувати інструменти (Інструкція, Частина 1, "Preparing the Tools")

Опис роботи:

1. *Підготовка ізольованого середовища (Docker):*

Для уникнення конфліктів в основній операційній системі та забезпечення відтворюваності збірки було використано Docker-контейнер на базі Ubuntu 24.04.

‘docker-compose.yml’:

```yaml
services:
  lab3-arm-env:
    image: ubuntu:20.04
    platform: linux/amd64
    container_name: lab3_Kopylets
    tty: true
    stdin_open: true
    privileged: true
    environment:
      - DEBIAN_FRONTEND=noninteractive
    volumes:
      - .:/project
      - work:/work
    working_dir: /project
    command: ["sleep", "infinity"]

volumes:
  work:
```

2. *Скрипт автоматичного встановлення залежностей*

Для налаштування середовища розроблено скрипт setup.sh. Щоб уникнути помилки multiple definition of yylloc , за рекомендацією було встановлено компілятор GCC версії 9 та призначено версією за замовчуванням. Скрипт також завантажує необхідні крос-компілятори ARM та клонує репозиторії U-Boot, Linux Kernel та BusyBox.

‘setup.sh’:

```bash
#!/bin/bash
set -euo pipefail

WORK=${WORK:-/work}
PROJECT=${PROJECT:-/project}
LOGS=$PROJECT/logs
mkdir -p "$WORK/src" "$LOGS"

if [ -f "$WORK/.setup_complete" ]; then
    echo "Середовище вже налаштовано. Запуск bash..."
```

```
    exec bash
fi
EABI_DIR=gcc-arm-8.3-2019.03-x86_64-arm-eabi
GNU_DIR=gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf
ARM_URL=https://developer.arm.com/-/media/Files/downloads/gnu-a/8.3-
2019.03/binrel
clone() {
    local url=$1 dir=$2
    shift 2
    if [ ! -d "$dir/.git" ]; then
        rm -rf "$dir.tmp"
        git clone "$@" "$url" "$dir.tmp"
        mv "$dir.tmp" "$dir"
    fi
}
get_toolchain() {
    local name=$1 f="$1.tar.xz" i
    [ -d "/opt/$name" ] && return 0
    for i in 1 2 3; do
        xz -t "$f" 2>/dev/null && break
        rm -f "$f"
        wget --tries=5 --retry-connrefused -O "$f" "$ARM_URL/$f" || true
    done
    xz -t "$f"
    tar xJf "$f" -C /opt/
}

do_setup() {
    export DEBIAN_FRONTEND=noninteractive
    echo "Оновлення системи та встановлення залежностей..."
    apt-get update
    apt-get install -y --no-install-recommends \
        ca-certificates git vim tree curl wget ccache make bc bison flex \
        xz-utils file bzip2 kmod cpio build-essential gcc-9 g++-9 \
        libncurses-dev libssl-dev libgnutls28-dev libmpc-dev libmpfr-dev \
        libgmp-dev uuid-dev python3 python3-dev python3-setuptools \
        python3-pyelftools swig qemu-system-arm

    echo "Налаштування GCC 9 за замовчуванням..."
    update-alternatives --install /usr/bin/gcc gcc /usr/bin/gcc-9 100
    update-alternatives --install /usr/bin/g++ g++ /usr/bin/g++-9 100
    update-alternatives --set gcc /usr/bin/gcc-9
    update-alternatives --set g++ /usr/bin/g++-9

    git config --global user.name "Daria Kopylets"
    git config --global user.email "kopylets.daria_io_44@edu.kpi.ua"
    ccache -M 5G

    cd "$WORK/src"

    echo "Завантаження компіляторів ARM..."
```

```
    get_toolchain "$EABI_DIR"
    get_toolchain "$GNU_DIR"

    echo "Клонування репозиторіїв..."
    clone https://source.denx.de/u-boot/u-boot.git u-boot
    clone https://git.kernel.org/pub/scm/linux/kernel/git/stable/linux-stable.git
linux-stable --depth 1 --branch linux-4.19.y
    clone https://git.busybox.net/busybox busybox --depth 1 --branch 1_31_stable

    echo "Підготовка U-Boot..."
    cd u-boot
    git am --abort 2>/dev/null || true
    LAST_TAG=$(git tag | grep -v rc | sort -V | tail -n 1)
    git checkout -f "$LAST_TAG"
    if ! curl -fsSL https://patchwork.ozlabs.org/series/130450/mbox/ | git am;
then
        git am --abort 2>/dev/null || true
        echo "Патч 130450 не застосовано до $LAST_TAG (для QEMU не критично)"
    fi
    cd "$WORK/src"

    echo "Збереження звіту про середовище..."
    {
        echo "== host gcc ==";                  gcc --version | head -1
        echo "== arm-eabi-gcc ==";              /opt/$EABI_DIR/bin/arm-eabi-gcc -
-version | head -1
        echo "== arm-linux-gnueabihf-gcc ==";   /opt/$GNU_DIR/bin/arm-linux-
gnueabihf-gcc --version | head -1
        echo "== qemu ==";                      qemu-system-arm --version | head
-1
        echo "== U-Boot ==";                    git -C u-boot log -1 --format='%h
%D'
        echo "== Linux kernel ==";              git -C linux-stable log -1 --
format='%h %D'
        echo "== BusyBox ==";                   git -C busybox log -1 --
format='%h %D'
    } > "$LOGS/env_check.txt" 2>&1
    cat "$LOGS/env_check.txt"
}
do_setup 2>&1 | tee -a "$LOGS/setup.log"
touch "$WORK/.setup_complete"
echo "Налаштування завершено."
exec bash
```

3. Збірка програмного забезпечення

Процес збірки автоматизовано за допомогою скрипта build.sh. Скрипт виконує:

1. Збірку завантажувача U-Boot.
2. Конфігурацію та збірку ядра Linux.

3. Збірку мінімальної кореневої файлової системи на базі BusyBox.
4. Наповнення директорій /boot, /lib, /etc та створення скрипта ініціалізації rcS.
5. Пакування файлової системи в архів rootfs.cpio.gz та запуск QEMU.

‘build.sh’:

```
#!/bin/bash
set -euo pipefail
WORK=${WORK:-/work}
PROJECT=${PROJECT:-/project}
LOGS=$PROJECT/logs
OUT=$PROJECT/out
SRC=$WORK/src
BB=$SRC/busybox
KERNEL=$SRC/linux-stable
EABI=/opt/gcc-arm-8.3-2019.03-x86_64-arm-eabi/bin
GNU=/opt/gcc-arm-8.3-2019.03-x86_64-arm-linux-gnueabihf/bin
CORES=$(nproc)
mkdir -p "$LOGS" "$OUT"
exec > >(tee "$LOGS/build.log") 2>&1

export ARCH=arm

echo ">>> Збірка U-Boot <<<"
if cd "$SRC/u-boot" \
   && PATH=$EABI:$PATH CROSS_COMPILE='ccache arm-eabi-' make
am335x_boneblack_defconfig \
   && PATH=$EABI:$PATH CROSS_COMPILE='ccache arm-eabi-' make -j"$CORES"; then
    echo "U-Boot: збірка успішна"
else
    echo "U-Boot не зібрався (для запуску в QEMU не критично)"
fi
cd "$WORK"

echo ">>> Збірка Linux Kernel <<<"
(
export PATH=$EABI:$PATH
export CROSS_COMPILE='ccache arm-eabi-'
cd "$KERNEL"
mkdir -p fragments
cat > fragments/bbb.cfg <<'EOF'
CONFIG_USB_ANNOUNCE_NEW_DEVICES=y
CONFIG_USB_EHCI_ROOT_HUB_TT=y
CONFIG_AM335X_PHY_USB=y
CONFIG_USB_MUSB_TUSB6010=y
CONFIG_USB_MUSB_OMAP2PLUS=y
CONFIG_USB_MUSB_HDRC=y
CONFIG_USB_MUSB_DSPS=y
CONFIG_USB_MUSB_AM35X=y
CONFIG_USB_CONFIGFS=y
```

```
CONFIG_NOP_USB_XCEIV=y
CONFIG_USB_HID=y
CONFIG_USB_HIDDEV=y
CONFIG_USB_SERIAL=y
CONFIG_USB_SERIAL_PL2303=y
CONFIG_USB_SERIAL_GENERIC=y
CONFIG_USB_SERIAL_SIMPLE=y
CONFIG_USB_SERIAL_FTDI_SIO=y
CONFIG_USB_ULPI=y
CONFIG_USB_ULPI_BUS=y
CONFIG_BRIDGE=y
CONFIG_OF_OVERLAY=y
EOF
./scripts/kconfig/merge_config.sh arch/arm/configs/multi_v7_defconfig
fragments/bbb.cfg
make HOSTCC=gcc-9 -j"$CORES" zImage modules am335x-boneblack.dtb
)

echo ">>> Збірка BusyBox <<<"
(
export PATH=$GNU:$PATH
export CROSS_COMPILE='ccache arm-linux-gnueabihf-'
cd "$BB"
make defconfig
make -j"$CORES"
make install
)

echo ">>> Формування rootfs <<<"
cd "$BB"
mkdir -p _install/{boot,dev,etc/init.d,lib,proc,root,sys/kernel/debug,tmp}

cat > _install/etc/init.d/rcS <<'EOF'
#!/bin/sh
mount -t sysfs none /sys
mount -t proc none /proc
mount -t debugfs none /sys/kernel/debug
echo /sbin/mdev > /proc/sys/kernel/hotplug
mdev -s
EOF
chmod +x _install/etc/init.d/rcS
ln -sf bin/busybox _install/init

cp "$KERNEL/arch/arm/boot/zImage" _install/boot/
cp "$KERNEL/arch/arm/boot/dts/am335x-boneblack.dtb" _install/boot/
cp "$KERNEL/System.map" _install/boot/
cp "$KERNEL/.config" _install/boot/config

(
export PATH=$EABI:$PATH
export CROSS_COMPILE='ccache arm-eabi-'
```

```
export INSTALL_MOD_PATH=$BB/_install
cd "$KERNEL"
make HOSTCC=gcc-9 modules_install
)

LIBC_DIR=$("$GNU/arm-linux-gnueabihf-gcc" -print-sysroot)/lib
cp -a "$LIBC_DIR"/*.so* _install/lib/

echo '$MODALIAS=.* root:root 660 @modprobe "$MODALIAS"' > _install/etc/mdev.conf
echo 'root:x:0:' > _install/etc/group
echo 'root:x:0:0:root:/root:/bin/sh' > _install/etc/passwd
echo 'root::10933:0:99999:7:::' > _install/etc/shadow
echo "nameserver 8.8.8.8" > _install/etc/resolv.conf

echo ">>> Пакування rootfs <<<"
(cd _install && find . | cpio -o -H newc | gzip > ../rootfs.cpio.gz)

echo ">>> Перевірка артефактів <<<"
{
    echo "== U-Boot =="
    ls -lh "$SRC/u-boot/MLO" "$SRC/u-boot/u-boot.img" 2>&1 || true
    echo "== Kernel =="
    ls -lh "$KERNEL/arch/arm/boot/zImage" "$KERNEL/arch/arm/boot/dts/am335x-
boneblack.dtb" 2>&1 || true
    echo "== BusyBox =="
    file "$BB/_install/bin/busybox"
    "$GNU/arm-linux-gnueabihf-readelf" -d "$BB/_install/bin/busybox" | grep
NEEDED || true
    echo "== Модулі ядра у rootfs =="
    ls "$BB/_install/lib/modules/" || true
    echo "== Вміст /boot у rootfs =="
    ls -lh "$BB/_install/boot/"
    echo "== Розмір rootfs =="
    du -sh "$BB/_install"
    ls -lh "$BB/rootfs.cpio.gz"
} 2>&1 | tee "$LOGS/artifacts.txt"

cp -f "$KERNEL/arch/arm/boot/zImage" "$KERNEL/arch/arm/boot/dts/am335x-
boneblack.dtb" "$BB/rootfs.cpio.gz" "$OUT/"
cp -f "$SRC/u-boot/MLO" "$SRC/u-boot/u-boot.img" "$OUT/" 2>/dev/null || true

echo "Збірку завершено. Запуск QEMU: bash run_qemu.sh test"
```

Також окремо створено скрипт, який запускає зібране ядро в емуляторі QEMU. У режимі test він автоматично вводить перевірочні команди

‘run_qemu.sh’:

```
#!/bin/bash
# bash run_qemu.sh        - інтерактивний запуск
# bash run_qemu.sh test   - автоматичний прогін команд
```

```
set -euo pipefail

WORK=${WORK:-/work}
PROJECT=${PROJECT:-/project}
LOGS=$PROJECT/logs
BB=$WORK/src/busybox
BOOT_WAIT=${BOOT_WAIT:-40}
mkdir -p "$LOGS"
QEMU=(qemu-system-arm
      -kernel "$BB/_install/boot/zImage"
      -initrd "$BB/rootfs.cpio.gz"
      -machine virt -nographic -m 512
      --append "root=/dev/ram0 rw console=ttyAMA0,115200 mem=512M")
if [ "${1:-}" != "test" ]; then
    exec "${QEMU[@]}"
fi
echo "Автотест QEMU (очікування завантаження ${BOOT_WAIT} с)..."
{
    sleep "$BOOT_WAIT"
    echo
    sleep 3
    for c in "uname -a" "ls -l" "dmesg | grep init" "busybox --help | head -15"
"poweroff"; do
        echo "$c"
        sleep 4
    done
    sleep 5
} | timeout 300 "${QEMU[@]}" 2>&1 | tr -d '\r' | tee "$LOGS/qemu_session.log" ||
true

echo "Журнал збережено: $LOGS/qemu_session.log"
```

4. *Тестування в QEMU*

uname -a - підтверджує версію ядра 4.19.325 і архітектуру armv7l, тобто завантажилося саме зібране ядро.

ls -l - показує структуру rootfs

dmesg | grep init - рядки “Trying to unpack rootfs image as initramfs”, “Freeing initrd memory” та “Run /init as init process” доводять, що rootfs розпаковано у RAM і керування передано /init.

busybox --help - підтверджує версію BusyBox 1.31.1.

poweroff - штатне вимкнення.

```
/ # uname -a
Linux (none) 4.19.325 #1 SMP Fri Oct 16 11:00:24 UTC 2026 armv7l GNU/Linux
/ # ls -l
total 0
drwxr-xr-x    2 root     root             0 Oct 16 11:08 bin
drwxr-xr-x    2 root     root             0 Oct 16 11:08 boot
drwxr-xr-x    3 root     root             0 Oct 16 11:25 dev
drwxr-xr-x    3 root     root             0 Oct 16 11:08 etc
lrwxrwxrwx    1 root     root            11 Oct 16 11:08 init -> bin/busybox
drwxr-xr-x    3 root     root             0 Oct 16 11:08 lib
lrwxrwxrwx    1 root     root            11 Oct 16 11:08 linuxrc -> bin/busybox
dr-xr-xr-x   91 root     root             0 Jan  1  1970 proc
drwxr-xr-x    2 root     root             0 Oct 16 11:08 root
drwxr-xr-x    2 root     root             0 Oct 16 11:08 sbin
dr-xr-xr-x   12 root     root             0 Oct 16 11:25 sys
drwxr-xr-x    2 root     root             0 Oct 16 11:08 tmp
drwxr-xr-x    4 root     root             0 Oct 16 11:08 usr
/ # dmesg | grep init
[    0.000000] Memory: 405356K/524288K available (12288K kernel code, 1618K rwdata, 4796K rodata, 2048K init, 39
4K bss, 53396K reserved, 65536K cma-reserved, 0K highmem)
[    0.000000]       .init : 0x(ptrval) - 0x(ptrval)   (2048 kB)
[    0.074682] devtmpfs: initialized
[    0.104143] pinctrl core: initialized pinctrl subsystem
[    0.239068] SCSI subsystem initialized
[    0.379131] Trying to unpack rootfs image as initramfs...
[    1.754115] Freeing initrd memory: 25140K
[    1.903530] SuperH (H)SCI(F) driver initialized
[    1.904954] msm_serial: driver initialized
[    1.905277] STMicroelectronics ASC driver initialized
[    1.906402] STM32 USART driver initialized
```

```
[    1.903530] SuperH (H)SCI(F) driver initialized
[    1.904954] msm_serial: driver initialized
[    1.905277] STMicroelectronics ASC driver initialized
[    1.906402] STM32 USART driver initialized
[    2.174439] Run /init as init process
/ # busybox --help | head -15
BusyBox v1.31.1 (2026-10-16 11:08:14 UTC) multi-call binary.
BusyBox is copyrighted by many authors between 1998-2015.
Licensed under GPLv2. See source distribution for detailed
copyright notices.

Usage: busybox [function [arguments]...]
   or: busybox --list[-full]
   or: busybox --show SCRIPT
   or: busybox --install [-s] [DIR]
   or: function [arguments]...

        BusyBox is a multi-call binary that combines many common Unix
        utilities into a single executable.  Most people will create a
        link to busybox for each function they wish to use and BusyBox
        will act like whatever it was invoked as.
/ # poweroff
/ # umount: can't unmount /: Invalid argument
swapoff: can't open '/etc/fstab': No such file or directory
The system is going down NOW!
Sent SIGTERM to all processes
Sent SIGKILL to all processes
Requesting system poweroff
[   60.494016] reboot: Power down
Журнал збережено: /project/logs/qemu_session.log
```

Висновки: Під час виконання лабораторної роботи було успішно розгорнуто ізольоване середовище для крос-компіляції під архітектуру ARM. Було зібрано завантажувач, ядро Linux та кореневу файлову систему BusyBox. Також було проведено успішне тестування зібраної системи у програмному емуляторі QEMU.